GE VERNOVA

GridOS SCADA

0.7.0

SCADA Client Developer Guide

Copyright

Copyright 2026, GE Vernova and/or its affiliates ("GE Vernova"). All Rights Reserved.

GE and the GE Monogram are trademarks of General Electric Company used under trademark license.

This document is the confidential and proprietary information of GE Vernova and may not be reproduced, transmitted, stored, or copied in whole or in part, or used to furnish information to others, without the prior written permission of GE Vernova.

Change History

Date	Change Description
2025-Aug-21	GridOS SCADA 0.4.0: • This is the initial release of SCADA Client documentation for SCADA Client 0.1.0.

Contents

1. Introduction **4**
1.1. Purpose 4
1.2. Key Classes 4
2. SCADA Client Developer Guide **5**
2.1. Overview 5
2.2. Module Build (pom.xml) 5
2.3. ScadaClient 5
2.3.1. Purpose 5
2.3.2. Responsibilities 6
2.3.3. Usage Flow 6
2.3.4. Configuration 7
2.4. HeartbeatScheduler 8
2.4.1. Purpose 8
2.4.2. Responsibilities 8
2.4.3. Usage Flow 8
2.4.4. Configuration 8
2.5. Typical Execution Flow 9
2.6. Integration Notes 9

Proprietary - See Copyright Page

1. Introduction

SCADA Client is responsible for client-side operations within the GridOS SCADA ecosystem. It contains implementations that manage communication with Kafka producers and maintain connection health through heartbeat scheduling.

This module is essential for real-time data publishing and system health monitoring.

The responsibilities of SCADA Client include:

- Sending SCADA object updates (for example, Analog, Status, and Accumulator) to SCADA Service using Kafka.
- Maintaining active connections with GridOS SCADA through scheduled heartbeat signals to Kafka topics.

1.1. Purpose

The SCADA Client module enables:

- Reliable publishing of SCADA object updates into Kafka topics consumed by SCADA Services.
- Continuous monitoring of connectivity with minimal latency.
- Integration with other SCADA components to ensure real-time system updates.

1.2. Key Classes

- `ScadaClient`: Handles the creation and sending of messages to Kafka.
- `HeartbeatScheduler`: Maintains active communication through periodic heartbeats.

Any data produced by SCADA or consumed from SCADA can use this library for serialization and deserialization, ensuring consistency, and validation.

Proprietary - See Copyright Page

2. SCADA Client Developer Guide

2.1. Overview

The SCADA Client module provides a Java API for communicating with SCADA services. It abstracts client applications from underlying protocol or infrastructure changes.

The responsibilities of SCADA Client include:

- Sending SCADA object updates to SCADA Service using Kafka.
- Issuing periodic heartbeat signals to maintain client connectivity.

This guide covers:

- Core classes - `ScadaClient` and `HeartbeatScheduler`.
- How they integrate within the SCADA Client.
- Configuration and usage instructions.

2.2. Module Build (pom.xml)

To use the SCADA Client in your project, add the following dependency:

```
<dependency>
  <groupId>com.gridos.scada</groupId>
  <artifactId>scada-client</artifactId>
  <version>0.1.0</version>
</dependency>
```

If using a private Maven repository:

```
<repositories>
    <repository>
        <id>gridos-internal</id>
        <url>https://nexus.gridos.com/repository/maven-releases/</url>
    </repository>
</repositories>
```

2.3. ScadaClient

2.3.1. Purpose

`ScadaClient` implements the Kafka `Producer` interface to publish SCADA data objects to Kafka topics.

Proprietary - See Copyright Page

2.3.2. Responsibilities

- Initialize a Kafka producer with application configuration.
- Serialize SCADA objects (for example, Analog, Status, and Accumulator).
- Send messages to the configured Kafka topic.
- Return a status indicating success or failure.

2.3.3. Usage Flow

1. Prepare a SCADA object update.
2. Call `sendUpdate()` of the ScadaClient.
3. Evaluate the returned status code.

Examples

```
import com.gridos.commons.scada.avro.v2.Analog;
import com.gridos.commons.scada.avro.v2.ScadaAnalog;
import com.gridos.commons.scada.avro.v2.ScadaStatus;
import com.gridos.commons.scada.avro.v2.Source;
import com.gridos.commons.scada.avro.v2.Status;

@Component
public class ScadaClientTest {
    private ScadaClient scadaClient;

    @Autowired
    public ScadaClientTest(ScadaClient scadaClient) {
        this.scadaClient = scadaClient;
    }

    public sendWithoutSiteCheck(){
        List<Status> statusList = new ArrayList<Status>();
        statusList.add(Status.newBuilder().setId("mrid of measurement")
                        .setName(null)
                        .setValue(2)
                        .setQuality(512L)
                        .setSource(Source.TELEMETERED.ordinal())
                        .setTime(System.currentTimeMillis())
                        .setSite(null)
                        .build());

        int status = scadaClient.sendUpdate(ScadaStatus.newBuilder().setData(statusList).build());
        if (status == SUCCESS) {
            log.info("Update sent successfully");
        } else {
            log.error("Update failed");
        }
    }
```

```java
    public sendWithSiteCheck() {
        List<Analog> analogList = new ArrayList<Analog>();
        analogList.add(Analog.newBuilder().setId("mrid of measurement")
                        .setName(null)
                        .setValue(12.345)
                        .setQuality(1024L)
                        .setSource(Source.STATE_ESTIMATOR.ordinal())
                        .setTime(System.currentTimeMillis())
                        .setSite(null)
                        .build());

        if (scadaClient.isSiteActive()) {
            int status = scadaClient.sendUpdate(ScadaAnalog.newBuilder().setData(analogList).build());
            if (status == SUCCESS) {
                log.info("Update sent successfully");
            } else {
                log.error("Update failed");
            }
        }
    }
}
```

2.3.4. Configuration

Topic names, bootstrap servers, and serializers are defined in `application.properties`.

```yaml
scada:
  id: "ScadaClient" /* Unique identifier for the SCADA client */
  source: "CALC_ENGINE" /* Source of the SCADA data */
  dbVersion: "0.0.1" /* Version of the database schema */
  kafka:
    bootstrapServers: localhost:9092 /* Kafka bootstrap server where SCADA consumer integrated */
    maxBlockMs: 100
  topics:
    analog: com.gridos.scada.inbound.analog
    accum: com.gridos.scada.inbound.accum
    status: com.gridos.scada.inbound.status
    limit: com.gridos.scada.inbound.limit
  site:
    name: default-site /* Name of the active/backup site */
    multiSite:
      enabled: false
    managerUrl: http://localhost:4123/v1/sites/ /* URL for the site manager */
```

2.4. HeartbeatScheduler

2.4.1. Purpose

`HeartbeatScheduler` keeps the SCADA Client connection alive by periodically sending heartbeat messages.

2.4.2. Responsibilities

- Schedule fixed-interval heartbeat tasks.
- Monitor connection health.
- Retry on failure.

2.4.3. Usage Flow

1. Initialize during SCADA Client startup.
2. Scheduler automatically sends heartbeat signals.
3. Monitor logs for failure events.

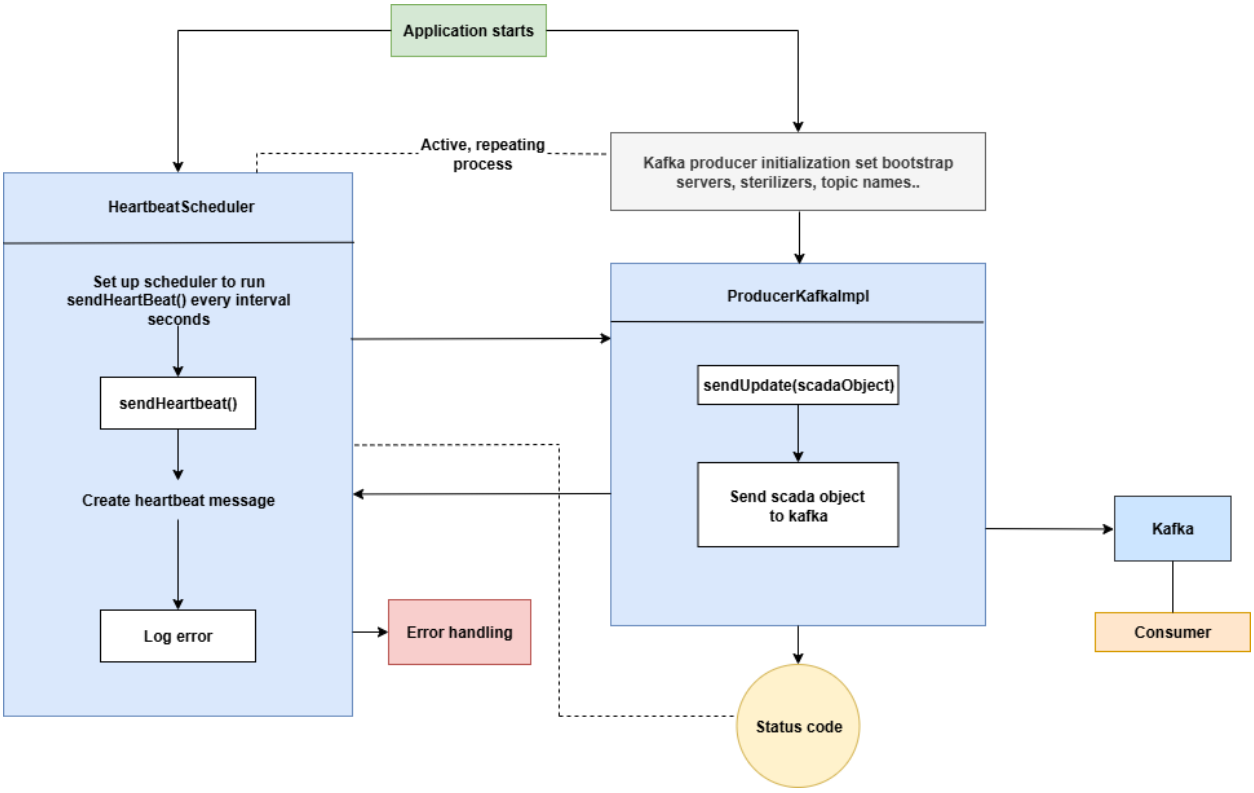
2.4.4. Configuration

Heartbeat intervals and retry parameters are set in `application.properties`.

```yaml
scada:
  heartbeat:
    topic: com.gridos.scada.inbound.heartbeat
    intervalSec: 8
    enabled: true
```


Proprietary - See Copyright Page

2.5. Typical Execution Flow



Sending measurement updates to SCADA consumer

1. `ScadaClient` sends an update to Kafka `Producer` using `sendUpdate(scadaObject)`.
2. Kafka `Producer` publishes the message to the `Kafka_Broker`.
3. `Kafka_Broker` acknowledges the status back to Kafka `Producer`.
4. Kafka `Producer` returns the status to `ScadaClient`.

   An internal scheduler runs every configurable number of seconds.

5. **ScadaClient** starts the `HeartbeatScheduler`.
6. When enabled, `HeartbeatScheduler` sends periodic heartbeat messages to a dedicated Kafka topic.
7. **Kafka_Broker** acknowledges the heartbeat back to `HeartbeatScheduler`.

2.6. Integration Notes

- Verify Kafka configuration before starting the client.
- Heartbeat failures may indicate network or broker issues.
- Changes to serialization formats must be coordinated with consumers.